1 Markov Assumption

1. Markov (Limited Horizon) Assumption: Transitions depend only on the current state. That is,
$$P(X_t|X_1 \dots X_{t-1}) \approx P(X_t|X_{t-1})$$
2. Output Independence Assumption: Probability of an output observation depends only on the current state and not on any other states or any other observations.
$$P(O_t|X_1 \dots X_t, \dots, X_T, O_1 \dots O_t \dots O_T) \approx P(O_t|X_t)$$

A Hidden Markov Model is a statistical model used to describe systems that transition between hidden states based on hidden states based on probability, where the hidden states are not directly visible but produce observations.

Considering the Markov Assumption, while it is still possible to calcuate the probability of a transition based on past states as well (ie. $X_1, X_2, \dots$), it is not practically feasible as our state space would explode, given that we have to account for the entire history of every previous state and every previous observation.

And without the Output Independence Assumption, we would not be able to infer just a specific hidden state from that observation, since we also have to account for the previous states. If we have M observations, we would have M inferrences for the first state, $M - 1$ for the second state, and so on.

2 HMM Artificial Data

2.1

Assuming the artificial data is the series of random observations, we need the set of N emitting hidden states, S, (with a special start/end state), an output alphabet of M observations, K, (with a special start and end symbol), the transition probabilities and the emission probabilities.

2.2

Based on the transition probabilities, we first generate a hidden sequence of states of some length, say X (excluding start and end states). Then for each state in the sequence, we use the emission probabilities to randomly emit an observation per state. The sequence of observations would be our generated random data.

2.3

This could be when the predictions result is of a fixed, known length. Then the transitions have to fit such that we hit the final state by that end point.

For example, when sequencing DNA, a "Stop Codon" acts as a physical final state that signals the end of a protein-coding region.

(Question for supervisor: Maybe we can talk a little more about this? I also thought about the idea below and I'm unsure if that makes sense, but it kind of does to me, and I think it also depends on the fixed length argument?)

what im thinking is suppose we have some states A, B, C and a final state F.

the second last state is A, and there is a really high probability we go into B. But the transition probablity from C to F is the highest, so it can be uncertain whether B or C is the final state?

3 Smoothing in HMMs

3.1

Smoothing involves adjusting probability parameters to handle events that did not appear in training data. (Parameter smoothing).

This is to ensure that transitions we have never seen (which would be assigned a probability of 0) would not break the model when deployed, as the probability of the entire sequence would become 0 since that specific transition is defined as "impossible".

However, smoothing can become unproductive if by smoothing, we introduce transitions that would not have been possible at all. This would violate the rules of the system we are trying to model.

3.2

Emissions probability would be a better candidate for smoothing over transition probability.

In biology, specific transitions are physically impossible, and smoothing transition probabilities would run the risk of predicting impossible chains.

If we consider the observations to be the observable amino acids produced by a hidden structural state, we are much more likley to have missing amino acid observations in training data than we are to have missing fundamental structural transitions.

4 Viterbi and Forward Algorithm

4.1

In the textbook, they specify an HMM with the following components:

- $Q = q_1 q_2 \dots q_N$: a set of N states
- $A = a_{11} \dots a_{ij} \dots a_{NN}$: a transition probability matrix A, with each a_{ij} representing the probability of moving from state i to state j such that

$$\forall i \sum_{j=1}^{N} a_{ij} = 1$$

- $B = b_i(o_t)$: a sequence of observation likelihoods (ie. emission probabilities), each expressing of an observation o_t drawn from a vocabulary $V = v_1, v_2, \dots v_V$ being generated from a state q_i.
- $\pi = \pi_1, \pi_2 \dots \pi_N$: an initial probability distribution over states. π_i is the probability that the Markov chain will start in state i. Some states j may have $\pi_j = 0$, meaning they cannot be initial states. Also,

$$\sum_{i=1}^{n} \pi_i = 1$$

The Forward Algorithm is to compute the likelihood of a particular observation sequence.

<u>Context</u>

Suppose we know what the hidden state sequence was. Then knowing that each hidden state produces a single observation, the likelihood of an observation sequence $O = o_1, o_2, \dots, o_T$

for the hidden state sequence $Q = q_1, q_2, \ldots q_T$ is

$$P(O|Q) = \prod_{i=1}^{T} P(o_i|q_i)$$

But we do not know what the hidden state sequence is. So we need to compute the probability of the observations by summing over all possible hidden state sequences, weighted by their probability. This would be

$$P(O, Q) = P(O|Q) \times P(Q) = \prod_{i=1}^{T} P(o_i|q_i) \times \prod_{i=1}^{T} P(q_i|q_{i-1})$$

which represents the joint probability of being in a particular hidden state sequence Q and generating a particular observation sequence O.

And the total probability of observation O can be computed by summing over all possible hidden state sequences:

$$P(O) = \sum_{Q} P(O, Q) = \sum_{Q} P(O|Q)P(Q)$$

We sum the probabilities as each of these hidden sequences can produce this observation sequence.

Forward Algorithm

The Forward Algorithm is a dynamic programming algorithm to efficiently compute the observation probability by summing over the probabilities of al possible hidden state paths that could generate the observation sequence by implicitly folding each of these paths into a single forward trellis.

Each cell of the forward algorithm trellis $\alpha_t(j)$ represents the probability of being in state j after seeing the first t observations given the automaton λ. (Question for supervisor: The automaton here is the FSM yes?) The value of each cell $\alpha_t(j)$ is computed by summing over the probabilities of every path that could lead us to the cell.

For a given state q_j at time t, the value $\alpha_t(j)$ is computed as

$$\alpha_t(j) = \sum_{i=1}^{N} \alpha_{t-1} a_{ij} b_j o(t)$$

where

- α_{t-1}(i) is the previous forward path probability from the previous time step
- a_{ij} is the transition probability from previous state q_i to current state q_j
- $b_j o(i)$ is the state observation likelihood of the observation symbol o_t given the current state j

4.2

Each state can be reached by multiple paths. Summing over the paths is to include all these paths in the probability calculation.

5 Parts of Speech tagging with HMM

5.1

X_a assigns "We" as a personal pronoun, "can" as an auxiliary verb and "fish" as a verb.

X_b assigns "We" as a personal pronoun, "can" as a verb and "fish" as a noun.

5.2

$$\begin{aligned} P(X_a) &= a_{03} \times a_{34} \times a_{41} \times a_{1f} \\ &= 0.60 \times 0.40 \times 0.73 \times 0.15 \\ &= 0.02628 \end{aligned}$$

$$\begin{aligned} P(X_b) &= a_{03} \times a_{31} \times a_{12} \times a_{2f} \\ &= 0.60 \times 0.40 \times 0.63 \times 0.20 \\ &= 0.03024 \end{aligned}$$

$$\begin{aligned} P(X_a, O) &= a_{03} \times b_3(\text{We}) \times a_{34} \times b_4(\text{can}) \times a_{41} \times b_1(\text{fish}) \times a_{1f} \\ &= 0.60 \times 1 \times 0.40 \times 1 \times 0.73 \times 0.89 \times 0.15 \\ &= 0.0233892 \end{aligned}$$

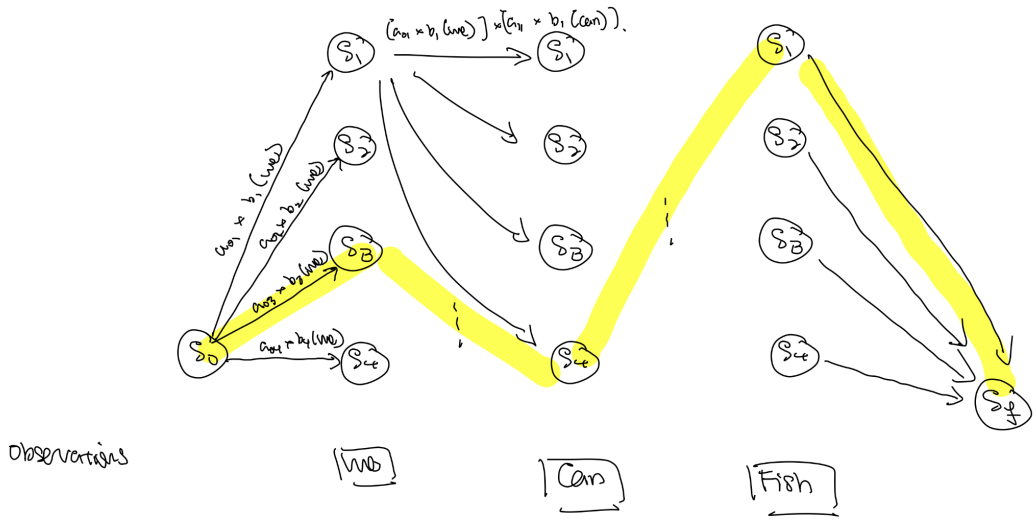
$$\begin{aligned} P(X_b, O) &= a_{03} \times b_3(\text{We}) \times a_{31} \times b_1(\text{can}) \times a_{12} \times b_2(\text{fish}) \times a_{2f} \\ &= 0.60 \times 1 \times 0.40 \times 0.10 \times 0.63 \times 0.75 \times 0.20 \\ &= 0.002268 \end{aligned}$$

$P(X_a, O)$ and $P(X_b, O)$ are used in the HMM. (Question for supervisor: This is referring to the Forward Algorithm yes? Not entirely sure what the question meant by "used in the HMM".)

5.3

The Viterbi algorithm is split into two parts: 1. The Forward Algorithm; 2. Backtracking.

In the forward algorithm, we compute the most likely path given the observation sequence O, and when we backtrack, we reconstruct the most probable parts of speech associated with O.



We move through time steps $t = 1, 2, 3$, and we let $v_t(j)$ be the Vertibi path probability for a current state j at time step t.

$$v_t(j) = \max_{i=1}^{N} \quad v_{t-1}(i) \cdot a_{ij} \cdot b_j(o_t)$$

<u>$t = 1$: "We"</u>

$$\begin{aligned}
v_1(s_1) &= a_{01} \cdot b_1(\text{“We”}) = 0.01 \cdot 0.01 = 0.0001 \\
v_1(s_2) &= a_{02} \cdot b_2(\text{“We”}) = 0.10 \cdot 0.01 = 0.001 \\
v_1(s_3) &= a_{03} \cdot b_3(\text{“We”}) = 0.60 \cdot 1 = 0.60 \\
v_1(s_4) &= a_{04} \cdot b_4(\text{“We”}) = 0.29 \cdot 0 = 0
\end{aligned}$$

This means we no longer need to consider paths that go from s_0 to s_4 and its subsequent pathing as this path is not possible (probability 0).

<u>$t = 2$: "can"</u>

$$\begin{aligned}
v_2(s_1) &= \max_{i=1}^{4} \quad v_1(i) \cdot a_{i1} \cdot b_1(\text{“can”}) = v_1(3) \cdot a_{31} \cdot b_1(\text{“can”}) = 0.60 \cdot 0.40 \cdot 0.10 = 0.024 \\
v_2(s_2) &= \max_{i=1}^{4} \quad v_1(i) \cdot a_{i2} \cdot b_2(\text{“can”}) = v_1(3) \cdot a_{32} \cdot b_2(\text{“can”}) = 0.60 \cdot 0.05 \cdot 0.24 = 0.0072 \\
v_2(s_3) &= \max_{i=1}^{4} \quad v_2(i) \cdot a_{i3} \cdot b_3(\text{“can”}) = v_1(3) \cdot a_{33} \cdot b_3(\text{“can”}) = 0.60 \cdot 0.05 \cdot 0 = 0 \\
v_2(s_4) &= \max_{i=1}^{4} \quad v_1(i) \cdot a_{i4} \cdot b_4(\text{“can”}) = v_1(3) \cdot a_{34} \cdot b_4(\text{“can”}) = 0.60 \cdot 0.40 \cdot 1 = 0.24
\end{aligned}$$

Paths from s_3 at $t = 2$ no longer need to be considered as well when we try to find the most probable path.

Also for backtracking, we keep a backpointer to state i at time $t - 1$ (which is $t = 1$ here) so we can reconstruct the path later.

<u>$t = 3$: "fish"</u>

$$\begin{aligned}
v_3(s_1) &= \max_{i=1}^{4} \quad v_2(i) \cdot a_{i1} \cdot b_1(\text{“fish”}) = v_2(4) \cdot a_{41} \cdot b_1(\text{“fish”}) = 0.24 \cdot 0.73 \cdot 0.89 = 0.155928 \\
v_3(s_2) &= \max_{i=1}^{4} \quad v_2(i) \cdot a_{i2} \cdot b_2(\text{“fish”}) = v_2(1) \cdot a_{12} \cdot b_2(\text{“fish”}) = 0.024 \cdot 0.63 \cdot 0.75 = 0.01134 \\
v_3(s_3) &= \max_{i=1}^{4} \quad v_2(i) \cdot a_{i3} \cdot b_3(\text{“fish”}) = \max_{i=1}^{4} \quad v_2(i) \cdot a_{i3} \cdot 0 = 0 \\
v_3(s_4) &= \max_{i=1}^{4} \quad v_2(i) \cdot a_{i4} \cdot b_4(\text{“fish”}) = \max_{i=1}^{4} \quad v_2(i) \cdot a_{i4} \cdot 0 = 0
\end{aligned}$$

<u>To end state</u>

$$v_4(s_f) = \max_{i=1}^{4} \quad v_3(i) \cdot a_{if} = v_3(1) \cdot a_{1f} = 0.0233892$$

At each time step t from 4 down to 1, we find the previous state that led to the highest probability.

So from the end state, we go back to s_1, where we go back to s_4, which goes back to s_3, and back to the start state.

Thus the most probable sequence of parts of speech given O is {personal pronoun, auxiliary verb, verb}

5.4

The model did arrive at the correct disambiguation. It did that through the Viterbi algorithm.

A main consideration is the fact that Viterbi algorithm considers the global path instead of local transitions. This means that we consider the entire path that gets us the highest probability, instead of local maximums which might not be the most probable path overall.

We also had the transition matrix A acting as "grammar", eliminating less probable sequences (eg. personal pronoun preceded by a noun, a_{32} has very low probability).

5.5

Emission probability answers: Given we are in state j, what is the chance we emit observation o? (This explains the denominator of the ratio below)

We can use the proportion of labels to estimate the emission probability matrix. We can calculate using this ratio:

$$b_i(o_k) = \frac{\text{Count}(s_j, o_k)}{\text{Count}(s_j)}$$

where $\text{Count}(s_j, o_k)$ is the number of times word o_k is labelled with state s_j, and $\text{Count}(s_j)$ is the total number of times state s_j appears in the sample.

5.6

(Question for supervisor: Didn't really understand this question, could we talk about this? Specifically I can't identify what the problem with "can" is in X_b.)

Suppose we train the emission probability matrix using labelled text. There is a chance, for example, we would not see "can" used as a verb at all, which would make the probability 0 instead of 0.10.

That means the model thinks it is impossible for "can" to be used as a verb, but it very much could be.

A potential solution is to smooth the probabilities, such that we account for such cases that we have not seen before.

6 Viterbi with higher order HMMs

6.1

For a N order HMM, the probability of the current state depends on the previous N states.

This means the previous N states matter in getting to the current state/observation. Suppose $N = 2$. Then if we consider the current state, there are N^2 possible previous state combinations that can get us to the current state.

So we need to keep N^N states for an N order HMM.

6.2

In 1st order HMM, for each of the N new states, we have to consider N old states to find the best path for each state. So we consider $N \times N = N^2$ transitions total.

Similarly for N order HMM, for each of the M states, we need to consider M^N previous states for each state. So we consider M^{N+1} transitions in total.

So the time complexity now goes from $O(T \cdot N^2)$ to $O(T \cdot M^{N+1})$.

(Question for supervisor: I feel the question variable naming is really confusing? But I can roughly understand the idea.)